Experiment No. 6
Implement Content-based recommendation system
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implement Content-based recommendation system

Objective: Able to design and implement a content based recommendation system.

Theory:
Content-based filtering system: Content-Based recommender system tries to guess the features or behaviour of a user given the item's features, he/she reacts positively to.

Movies	User 1	User 2	User 3	User 4	Action	Comedy
Item 1	1		4	5	Yes	No
Item 2	5	4	1	2	No	Yes
Item 3	4	4		3	Yes	Yes
Item 4	2	2	4	4	No	Yes

The last two columns Action and Comedy Describe the Genres of the movies. Now, given these genres, we can know which users like which genre, as a result, we can obtain features corresponding to that particular user, depending on how he/she reacts to movies of that genre. Once, we know the likings of the user we can embed him/her in an embedding space using the feature vector generated and recommend him/her according to his/her choice. During recommendation, the similarity metrics (We will talk about it in a bit) are calculated from the item's feature vectors and the user's preferred feature vectors from his/her previous records. Then, the top few are recommended.
Content-based filtering does not require other users' data during recommendations to one user.

Implicit Feedback: The user's likes and dislikes are noted and recorded on the basis of his/her actions like clicks, searches, and purchases. They are found in abundance, but negative feedback is not found.

Explicit Feedback: The user specifies his/her likes or dislikes by actions like reacting to an item or rating it. It has both positive and negative feedback but less in number.

Implementation:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Sample Movie Dataset with Features (Content)
movies = pd.DataFrame({
    'movie_id': [1, 2, 3, 4, 5],
    'title': ['Movie1', 'Movie2', 'Movie3', 'Movie4', 'Movie5'],
    'Action': [1, 0, 1, 0, 1],
    'Comedy': [0, 1, 1, 0, 0]
})

print("Movies Dataset:")
print(movies)

# Sample User Ratings (Explicit Feedback)
ratings = pd.DataFrame({
    'movie_id': [1, 2, 3],
    'rating': [5, 3, 4]
})

print("\nUser Ratings:")
print(ratings)

# Step 1: Merge ratings with movie features
user_data = pd.merge(ratings, movies, on='movie_id')

# Step 2: Create User Profile (Weighted feature vector)
user_profile = {
    'Action': (user_data['Action'] * user_data['rating']).sum(),
    'Comedy': (user_data['Comedy'] * user_data['rating']).sum()
```

```
}

user_profile_df = pd.DataFrame([user_profile])

print("\nUser Profile (Feature Preferences):")
print(user_profile_df)

# Step 3: Compute similarity between user profile and all movies
movie_features = movies[['Action', 'Comedy']]

similarity_scores = cosine_similarity(user_profile_df, movie_features)

movies['similarity'] = similarity_scores[0]

# Step 4: Recommend movies (excluding already rated)
recommended_movies = movies[~movies['movie_id'].isin(ratings['movie_id'])]
recommended_movies = recommended_movies.sort_values(by='similarity',
ascending=False)

print("\nRecommended Movies:")
print(recommended_movies[['title', 'similarity']])
```

Conclusion:

In this experiment, a Content-Based Recommendation System was implemented using Python. The system utilized item features such as genres (Action and Comedy) along with user ratings to build a user profile representing individual preferences. Cosine similarity was used to measure the similarity between the user profile and movie features, enabling personalized recommendations. Unlike collaborative filtering, this method does not rely on other users' data and works effectively even with a single user's history. Thus, content-based filtering provides accurate and personalized recommendations based on user interests.